Plaza Works App - KPI Module Test Suite

Instructions for Replit

Run these tests after completing all 5 KPI Module phases (A through E). Paste the prompt at the bottom of this document into Replit to have it run all tests automatically.

Test Configuration

Test Data Setup

```
const KPI_TEST_CONFIG = {
  // Staff with known costs for calculation verification
  testPlumber: {
    email: 'plumber1@test.plazaworks.com',
    hourlyLoadedCost: 84.59,  // $636.44/day ÷ 7.5 hours
    dailyLoadedCost: 636.44,
    weeklyTarget: 10000,
    dailyTarget: 2000,
    salesPhase: 1
  },

  testPlumber2: {
    email: 'plumber2@test.plazaworks.com',
    hourlyLoadedCost: 93.67,  // $702.50/day ÷ 7.5 hours
    dailyLoadedCost: 702.50,
    weeklyTarget: 10000,
    dailyTarget: 2000,
    salesPhase: 2
  },

  testApprentice: {
    email: 'apprentice@test.plazaworks.com',
    hourlyLoadedCost: 53.97,  // $404.80/day ÷ 7.5 hours
    dailyLoadedCost: 404.80,
    weeklyTarget: null,  // Apprentices don't have labor targets
    dailyTarget: null
  },

  // Team configurations for target testing
  teamConfigs: {
    '1P+1A': {
```

```
      monthlyRevenue: 64950,
      monthlyProfit: 3001,
      weeklyLabor: 10000
    },
    '2P+2A': {
      monthlyRevenue: 129900,
      monthlyProfit: 22328,
      weeklyLabor: 20000
    },
    '3P+2A': {
      monthlyRevenue: 194850,
      monthlyProfit: 51120,
      weeklyLabor: 30000
    }
  },

  // Bonus tiers for verification
  phase2Bonus: [
    { min: 10000, max: 12000, bonus: 0 },
    { min: 12001, max: 14000, bonus: 300 },
    { min: 14001, max: 16000, bonus: 500 },
    { min: 16001, max: 18000, bonus: 750 },
    { min: 18001, max: Infinity, bonus: 1000 }
  ],

  phase3Bonus: [
    { min: 10000, max: 12000, bonus: 0 },
    { min: 12001, max: 15000, bonus: 400 },
    { min: 15001, max: 18000, bonus: 700 },
    { min: 18001, max: 22000, bonus: 1000 },
    { min: 22001, max: Infinity, bonus: 1500 }
  ],

  // Alert thresholds
  alertThresholds: {
    dailyLaborWarning: 1500,
    weeklyLaborWarning: 8000,
    closeRateWarning: 50,
    leadCostDrainageWarning: 80,
    leadCostBathroomWarning: 150,
    invoiceOverdueDays: 14,
    cashReserveWeeks: 1
  }
};
```

Phase A Tests: Database and Data Layer

A.1 KPI Tables Created

```
describe('KPI Database Tables', () => {
  test('A.1.1 - kpi_daily_snapshots table exists', async () => {
    // Query database schema
    // Verify table exists with columns:
    // id, user_id, snapshot_date, labor_revenue, hours_logged, jobs_completed,
    // quotes_sent_value, quotes_sent_count, quotes_accepted_value,
quotes_accepted_count,
    // target_labor, target_met, created_at
  });

  test('A.1.2 - kpi_weekly_snapshots table exists', async () => {
    // Verify table exists with columns:
    // id, user_id, week_start, week_end, labor_revenue, hours_logged, jobs_completed,
    // quotes_sent_value, close_rate, target_labor, target_met, days_target_met, created_at
  });

  test('A.1.3 - kpi_monthly_snapshots table exists', async () => {
    // Verify table exists with columns:
    // id, user_id (nullable), month, labor_revenue, materials_revenue, total_revenue,
    // total_costs, net_profit, target_revenue, target_profit, variance_revenue,
    // variance_profit, jobs_completed, jobs_started, average_close_rate, created_at
  });

  test('A.1.4 - kpi_targets table exists', async () => {
    // Verify table exists with columns:
    // id, team_config, daily_labor_target, weekly_labor_target, monthly_revenue_target,
    // monthly_profit_target, close_rate_target, is_active, effective_from, created_at,
updated_at
  });

  test('A.1.5 - kpi_alerts_log table exists', async () => {
    // Verify table exists with columns:
    // id, alert_type, severity, user_id, triggered_at, message, data (JSON),
    // acknowledged, acknowledged_by, acknowledged_at
  });

  test('A.1.6 - tradesman_bonus_periods table exists', async () => {
    // Verify table exists with columns:
    // id, user_id, period_start, period_end, closed_sales_labor_value, sales_phase,
    // bonus_tier, bonus_amount, approved_by, approved_at, paid, paid_at
  });

  test('A.1.7 - phase_progression_checklist table exists', async () => {
    // Verify table exists with columns:
```

```
    // id, user_id, from_phase, to_phase, checklist_item, completed, completed_at,
    // verified_by, notes
  });

  test('A.1.8 - user_phase_log table exists', async () => {
    // Verify table exists with columns:
    // id, user_id, previous_phase, new_phase, changed_by, changed_at, notes
  });
});
```

A.2 User Table Extensions

```
describe('User Table KPI Fields', () => {
  test('A.2.1 - hourly_cost_loaded field exists', async () => {
    // Query user record
    // Verify hourly_cost_loaded field exists and accepts decimal
  });

  test('A.2.2 - daily_cost_loaded field exists', async () => {
    // Verify daily_cost_loaded field exists
  });

  test('A.2.3 - sales_phase field exists with default', async () => {
    // Verify sales_phase field exists
    // Verify default value is 1
  });

  test('A.2.4 - phase_start_date field exists', async () => {
    // Verify phase_start_date field exists
  });

  test('A.2.5 - weekly_labor_target field exists with default', async () => {
    // Verify field exists
    // Verify default value is 10000
  });

  test('A.2.6 - daily_labor_target field exists with default', async () => {
    // Verify field exists
    // Verify default value is 2000
  });
});
```

A.3 Seed Data

```
describe('KPI Seed Data', () => {
  test('A.3.1 - 1P+1A target config exists', async () => {
    // Query kpi_targets for team_config = '1P+1A'
```

```
    // Verify: daily_labor_target = 2000, weekly = 10000, monthly_revenue = 64950,
monthly_profit = 3001
  });

  test('A.3.2 - 2P+2A target config exists', async () => {
    // Verify: monthly_revenue = 129900, monthly_profit = 22328
  });

  test('A.3.3 - 3P+2A target config exists', async () => {
    // Verify: monthly_revenue = 194850, monthly_profit = 51120
  });

  test('A.3.4 - Close rate target is 70%', async () => {
    // Verify all configs have close_rate_target = 70
  });
});
```

Phase B Tests: Core KPI Calculations

B.1 Daily Labor Revenue Calculation

```
describe('Daily Labor Revenue', () => {
  test('B.1.1 - Calculate daily labor from completed tasks', async () => {
    // Create job with labor line items totaling $2,500
    // Complete tasks for today
    // Query daily labor revenue
    // Verify equals $2,500
  });

  test('B.1.2 - Daily labor aggregates multiple jobs', async () => {
    // Create 2 jobs, each with $1,200 labor
    // Complete both today
    // Verify daily total = $2,400
  });

  test('B.1.3 - Daily labor only counts today', async () => {
    // Create job completed yesterday with $2,000 labor
    // Create job completed today with $1,500 labor
    // Verify today's total = $1,500 only
  });

  test('B.1.4 - Daily labor per tradesman', async () => {
    // Create jobs assigned to different tradesmen
    // Verify each tradesman's daily total is separate
  });
```

```
  test('B.1.5 - Target met flag when >= $2,000', async () => {
    // Create $2,000 in labor for today
    // Verify target_met = true in daily snapshot
  });

  test('B.1.6 - Target not met when < $2,000', async () => {
    // Create $1,800 in labor for today
    // Verify target_met = false
  });
});
```

B.2 Weekly Labor Revenue Calculation

```
describe('Weekly Labor Revenue', () => {
  test('B.2.1 - Weekly labor sums daily revenues', async () => {
    // Create daily snapshots: Mon $2000, Tue $1800, Wed $2200, Thu $2000, Fri $2000
    // Calculate weekly total
    // Verify equals $10,000
  });

  test('B.2.2 - Weekly target met when >= $10,000', async () => {
    // Create week with $10,500 total
    // Verify target_met = true
  });

  test('B.2.3 - Days target met count accurate', async () => {
    // Create week: 3 days at $2,000+, 2 days below
    // Verify days_target_met = 3
  });

  test('B.2.4 - Week boundaries correct', async () => {
    // Verify week starts Monday, ends Sunday
    // Verify labor from previous week not included
  });
});
```

B.3 Close Rate Calculation

```
describe('Close Rate', () => {
  test('B.3.1 - Close rate formula correct', async () => {
    // Create 10 quotes sent, 7 accepted
    // Verify close_rate = 70%
  });

  test('B.3.2 - Close rate per tradesman', async () => {
    // Tradesman A: 10 sent, 8 accepted (80%)
```

```
    // Tradesman B: 10 sent, 5 accepted (50%)
    // Verify individual rates calculated correctly
  });

  test('B.3.3 - Declined quotes count in denominator', async () => {
    // 10 sent, 5 accepted, 3 declined, 2 pending
    // Close rate should be 5/8 = 62.5% (only count responded)
    // OR 5/10 = 50% (count all sent)
    // Verify which formula is used and it's consistent
  });

  test('B.3.4 - Zero quotes handled', async () => {
    // No quotes sent
    // Verify no division by zero error
    // Verify close_rate = 0 or null
  });
});
```

B.4 Job Profitability Calculation

```
describe('Job Profitability', () => {
  test('B.4.1 - Job profit = revenue - labor cost', async () => {
    // Job: $2,955 labor revenue
    // Hours logged: 12 hours at $84.59/hr = $1,015.08
    // Expected profit: $2,955 - $1,015.08 = $1,939.92
    // Verify calculation
  });

  test('B.4.2 - Uses fully loaded hourly rate', async () => {
    // Verify calculation uses hourly_cost_loaded, not base rate
  });

  test('B.4.3 - Daily profit rate calculation', async () => {
    // Job profit: $1,940
    // Actual duration: 1.5 days
    // Daily rate: $1,940 / 1.5 = $1,293.33
    // Verify calculation
  });

  test('B.4.4 - Multiple staff on job', async () => {
    // Plumber: 8 hours at $84.59 = $676.72
    // Apprentice: 8 hours at $53.97 = $431.76
    // Total labor cost: $1,108.48
    // Verify both costs included
  });
});
```

B.5 Duration Variance Calculation

```
describe('Duration Variance', () => {
  test('B.5.1 - Variance = actual - quoted', async () => {
    // Quoted: 1.5 days
    // Actual: 2 days
    // Variance: +0.5 days
    // Verify calculation
  });

  test('B.5.2 - Negative variance when faster', async () => {
    // Quoted: 2 days
    // Actual: 1 day
    // Variance: -1 day
    // Verify shows as negative
  });

  test('B.5.3 - Actual duration excludes non-work days', async () => {
    // Started Friday, completed Monday
    // Verify weekend not counted
  });
});
```

B.6 Schedule Capacity Calculation

```
describe('Schedule Capacity', () => {
  test('B.6.1 - Percentage booked calculation', async () => {
    // 5 available days, 3 booked
    // Capacity: 60%
    // Verify calculation
  });

  test('B.6.2 - Capacity per period accurate', async () => {
    // This week: 4/5 = 80%
    // Next week: 2/5 = 40%
    // Verify each period calculated separately
  });

  test('B.6.3 - Multiple tradesmen aggregate', async () => {
    // 2 tradesmen, 10 total available days
    // 7 days booked
    // Team capacity: 70%
    // Verify team-level calculation
  });

  test('B.6.4 - Per tradesman capacity', async () => {
    // Tradesman A: 5/5 = 100%
    // Tradesman B: 2/5 = 40%
```

```
    // Verify individual calculations
  });
});
```

B.7 API Endpoints

```
describe('KPI API Endpoints', () => {
  test('B.7.1 - GET /api/v1/kpi/dashboard/daily returns data', async () => {
    // Call endpoint with auth
    // Verify 200 response
    // Verify data includes: labor_revenue, hours_logged, jobs_completed, target_met
  });

  test('B.7.2 - GET /api/v1/kpi/dashboard/weekly returns data', async () => {
    // Verify includes: weekly totals, close_rate, days_target_met
  });

  test('B.7.3 - GET /api/v1/kpi/dashboard/monthly returns data', async () => {
    // Verify includes: revenue, costs, profit, variance
  });

  test('B.7.4 - GET /api/v1/kpi/dashboard/owner requires owner role', async () => {
    // Call as plumber
    // Verify 403 forbidden
  });

  test('B.7.5 - GET /api/v1/kpi/tradesman/{id}/summary returns data', async () => {
    // Verify includes: all KPI metrics for that tradesman
  });

  test('B.7.6 - GET /api/v1/kpi/pipeline returns quote pipeline', async () => {
    // Verify includes: open_quotes_value, expected_close_value
  });

  test('B.7.7 - GET /api/v1/kpi/capacity returns capacity data', async () => {
    // Verify includes: this_week, next_week, plus_2, plus_3, plus_4 percentages
  });
});
```

Phase C Tests: Dashboard UI

C.1 Business Owner Dashboard

```
describe('Owner Dashboard', () => {
  test('C.1.1 - Daily view loads', async () => {
```

```
    // Log in as owner
    // Navigate to dashboard
    // Verify daily view is default
  });

  test('C.1.2 - Per-tradesman cards displayed', async () => {
    // Verify card for each active tradesman
    // Each card shows: name, labor revenue, hours, jobs, quotes
  });

  test('C.1.3 - Status indicators correct', async () => {
    // Tradesman at $2,100 (above target) → Green
    // Tradesman at $1,800 (75-99%) → Amber
    // Tradesman at $1,200 (below 75%) → Red
    // Verify colors match status
  });

  test('C.1.4 - Quick summary cards present', async () => {
    // Verify: Total team revenue, Total quotes, Jobs completed, Red flags count
  });

  test('C.1.5 - Toggle to weekly view', async () => {
    // Click weekly tab
    // Verify weekly metrics displayed
    // Verify includes: close rate, cash position, pipeline
  });

  test('C.1.6 - Toggle to monthly view', async () => {
    // Click monthly tab
    // Verify monthly P&L displayed
    // Verify includes: revenue vs target, profit variance
  });

  test('C.1.7 - Cash position displayed', async () => {
    // Verify shows: current balance, weeks of coverage
  });

  test('C.1.8 - Pipeline health shown', async () => {
    // Verify shows: open quotes value, weeks of pipeline
  });

  test('C.1.9 - Red flags highlighted', async () => {
    // Create condition that triggers red flag
    // Verify flag appears prominently on dashboard
  });
});
```

C.2 Tradesman Dashboard

```
describe('Tradesman Dashboard', () => {
  test('C.2.1 - Shows own daily numbers', async () => {
    // Log in as plumber1
    // Verify see: labor revenue, hours logged, jobs progressed
  });

  test('C.2.2 - Shows own weekly numbers', async () => {
    // Verify: weekly labor, weekly quotes, close rate
  });

  test('C.2.3 - Progress bar for weekly target', async () => {
    // At $7,000 of $10,000 target
    // Verify progress bar shows 70%
  });

  test('C.2.4 - Calendar heat map displays', async () => {
    // Verify last 4 weeks shown
    // Verify days colored by performance
  });

  test('C.2.5 - Current sales phase shown', async () => {
    // Verify displays: "Phase 1" or "Phase 2" etc.
  });

  test('C.2.6 - Streak counter works', async () => {
    // 5 consecutive days hitting target
    // Verify shows "5 day streak"
  });

  test('C.2.7 - Cannot see other tradesmen data', async () => {
    // Verify no access to other users' KPIs
  });

  test('C.2.8 - Trend indicators shown', async () => {
    // This week higher than last
    // Verify up arrow displayed
  });
});
```

C.3 Apprentice Dashboard

```
describe('Apprentice Dashboard', () => {
  test('C.3.1 - Shows daily hours', async () => {
    // Log in as apprentice
    // Verify shows: hours logged today vs 7.5 target
  });
```

```
  test('C.3.2 - Shows weekly hours', async () => {
    // Verify shows: total weekly hours vs 37.5 target
  });

  test('C.3.3 - Shows assigned tradesman', async () => {
    // Verify displays who they're assigned to
  });

  test('C.3.4 - Simplified view', async () => {
    // Verify no financial data shown
    // Verify no quote/close rate metrics
  });
});
```

C.4 Marketing Dashboard

```
describe('Marketing Dashboard', () => {
  test('C.4.1 - Lead counts by type shown', async () => {
    // Verify shows: drainage leads, bathroom leads
  });

  test('C.4.2 - Lead cost metrics displayed', async () => {
    // Verify shows: cost per drainage lead, cost per bathroom lead
  });

  test('C.4.3 - Lead cost thresholds indicated', async () => {
    // Drainage at $60 → Green (under $80)
    // Drainage at $90 → Red (over $80)
    // Verify indicators
  });

  test('C.4.4 - Response time average shown', async () => {
    // Verify shows average response time
  });

  test('C.4.5 - Pipeline vs capacity displayed', async () => {
    // Verify shows comparison of lead volume to team capacity
  });
});
```

C.5 Dashboard Responsiveness

```
describe('Dashboard Mobile', () => {
  test('C.5.1 - Owner dashboard works on mobile', async () => {
    // View at mobile width
    // Verify all elements accessible
```

```
  });

  test('C.5.2 - Tradesman dashboard works on mobile', async () => {
    // Verify key metrics visible
    // Verify progress bar renders
  });

  test('C.5.3 - Data loads efficiently', async () => {
    // Measure load time
    // Verify under 3 seconds
  });
});
```

Phase D Tests: Alerts and Reports

D.1 Alert Triggers

```
describe('Alert Triggers', () => {
  test('D.1.1 - Daily target warning at $1,500', async () => {
    // Create $1,400 labor for tradesman today
    // Verify daily_target_warning alert triggered
    // Verify sent to tradesman and owner
  });

  test('D.1.2 - Free site visit alert', async () => {
    // Create site visit with $0 charge
    // Verify free_site_visit alert triggered
    // Verify sent to owner
  });

  test('D.1.3 - Cash critical alert', async () => {
    // Set cash balance below 1 week wages
    // Verify cash_critical alert triggered
    // Verify SMS sent to owner
  });

  test('D.1.4 - Job over duration alert', async () => {
    // Complete job 1+ day over quoted
    // Verify job_over_duration alert triggered
  });

  test('D.1.5 - Invoice overdue alert at 14 days', async () => {
    // Create invoice 15 days past due
    // Verify invoice_overdue alert triggered
  });
```

```
  test('D.1.6 - Weekly target missed alert', async () => {
    // Two consecutive weeks under $8,000
    // Verify weekly_target_missed alert triggered
  });

  test('D.1.7 - Close rate low alert', async () => {
    // Two consecutive weeks under 50% close rate
    // Verify close_rate_low alert triggered
  });

  test('D.1.8 - Pipeline low alert', async () => {
    // Pipeline value covers less than 1 week
    // Verify pipeline_low alert triggered
  });

  test('D.1.9 - Lead cost high alert', async () => {
    // Drainage lead cost exceeds $80
    // Verify alert triggered to marketing
  });
});
```

D.2 Alert Management

```
describe('Alert Management', () => {
  test('D.2.1 - GET /api/v1/kpi/alerts returns alerts', async () => {
    // Verify returns list of active alerts
  });

  test('D.2.2 - Alerts filterable by acknowledged', async () => {
    // Filter acknowledged=false
    // Verify only unacknowledged returned
  });

  test('D.2.3 - Acknowledge alert', async () => {
    // POST /api/v1/kpi/alerts/{id}/acknowledge
    // Verify acknowledged=true, acknowledged_by, acknowledged_at set
  });

  test('D.2.4 - Alerts logged to kpi_alerts_log', async () => {
    // Trigger an alert
    // Verify record created in kpi_alerts_log
  });

  test('D.2.5 - Alert severity levels correct', async () => {
    // cash_critical → severity: critical
    // daily_target_warning → severity: warning
```

```
    // Verify correct severity assigned
  });
});
```

D.3 Scheduled Jobs

```
describe('Scheduled Jobs', () => {
  test('D.3.1 - Daily snapshot finalize runs at 6 PM', async () => {
    // Check scheduled job exists
    // Manually trigger
    // Verify daily snapshots created for all tradesmen
  });

  test('D.3.2 - Daily report generates at 6:05 PM', async () => {
    // Trigger daily report
    // Verify report created
    // Verify sent to owner
  });

  test('D.3.3 - Hours reminder sent at 6 PM', async () => {
    // Create tradesman with no hours logged today
    // Trigger hours reminder
    // Verify reminder notification sent
  });

  test('D.3.4 - Weekly snapshot generates Friday 5 PM', async () => {
    // Trigger weekly snapshot
    // Verify weekly aggregates created
  });

  test('D.3.5 - Weekly report generates Friday 5:05 PM', async () => {
    // Trigger weekly report
    // Verify report created
    // Verify sent to owner and sales coach
  });

  test('D.3.6 - Monthly snapshot generates 1st of month', async () => {
    // Trigger monthly snapshot
    // Verify monthly aggregates created
  });

  test('D.3.7 - Monthly report generates 1st of month', async () => {
    // Trigger monthly report
    // Verify PDF generated
    // Verify sent to owner
  });
});
```

D.4 Report Contents

```
describe('Report Contents', () => {
  test('D.4.1 - Daily report includes per-tradesman data', async () => {
    // Generate daily report
    // Verify includes: each tradesman's revenue, hours, jobs
  });

  test('D.4.2 - Daily report includes red flags', async () => {
    // Create alert condition
    // Generate daily report
    // Verify alert mentioned in report
  });

  test('D.4.3 - Weekly report includes close rate', async () => {
    // Generate weekly report
    // Verify close rate analysis included
  });

  test('D.4.4 - Weekly report includes pipeline status', async () => {
    // Verify open quotes, expected close value
  });

  test('D.4.5 - Weekly report includes capacity', async () => {
    // Verify next 4 weeks capacity shown
  });

  test('D.4.6 - Monthly report includes P&L vs target', async () => {
    // Verify revenue, costs, profit vs targets
    // Verify variance calculations
  });

  test('D.4.7 - Reports exportable as PDF', async () => {
    // POST /api/v1/kpi/reports/export with format=pdf
    // Verify PDF generated
  });

  test('D.4.8 - Reports exportable as CSV', async () => {
    // POST /api/v1/kpi/reports/export with format=csv
    // Verify CSV generated
  });
});
```

Phase E Tests: Sales Phase and Bonus

E.1 Phase Progression Display

```
describe('Phase Progression Display', () => {
  test('E.1.1 - View tradesman phase screen', async () => {
    // Navigate to tradesman's phase progression
    // Verify current phase displayed
    // Verify weeks at phase shown
  });

  test('E.1.2 - Phase timeline displays history', async () => {
    // Tradesman who has progressed phases
    // Verify timeline shows: Phase 1 start → Phase 2 start → current
  });

  test('E.1.3 - Checklist items displayed', async () => {
    // View Phase 1 → 2 checklist
    // Verify all 4 items shown:
    // - 4+ weeks hitting $10k weekly
    // - Completed sales coaching
    // - Phone qualification skills
    // - Owner sign-off
  });

  test('E.1.4 - Phase 2 → 3 checklist different', async () => {
    // View Phase 2 → 3 checklist
    // Verify items:
    // - 8+ weeks at Phase 2
    // - $50k+ drainage closed
    // - Shadowed 3+ bathroom sales
    // - Owner sign-off
  });
});
```

E.2 Checklist Functionality

```
describe('Checklist Functionality', () => {
  test('E.2.1 - Owner can check items', async () => {
    // Log in as owner
    // Toggle checklist item
    // Verify saved to database
  });

  test('E.2.2 - Checked items show timestamp', async () => {
    // Check an item
    // Verify completed_at timestamp saved
```

```
  });

  test('E.2.3 - Items can be unchecked', async () => {
    // Uncheck previously checked item
    // Verify completed=false, timestamp cleared
  });

  test('E.2.4 - Non-owner cannot modify checklist', async () => {
    // Log in as tradesman
    // Attempt to toggle item
    // Verify access denied
  });

  test('E.2.5 - Auto-check weeks at target', async () => {
    // Create 4 weeks of snapshots hitting $10k
    // Verify "4+ weeks" item auto-checked or indicated as met
  });
});
```

E.3 Phase Advancement

```
describe('Phase Advancement', () => {
  test('E.3.1 - Advance button disabled when incomplete', async () => {
    // Checklist with 2/4 items complete
    // Verify "Advance Phase" button disabled
  });

  test('E.3.2 - Advance button enabled when complete', async () => {
    // Complete all checklist items
    // Verify button becomes active
  });

  test('E.3.3 - Advance phase updates user record', async () => {
    // Click advance
    // Verify user.sales_phase incremented
    // Verify user.phase_start_date updated
  });

  test('E.3.4 - Phase change logged', async () => {
    // Advance phase
    // Verify record in user_phase_log
    // Verify includes: previous_phase, new_phase, changed_by, timestamp
  });

  test('E.3.5 - Notification sent on phase change', async () => {
    // Advance phase
    // Verify tradesman receives notification
```

```
  });

  test('E.3.6 - Cannot advance past Phase 3', async () => {
    // Tradesman at Phase 3
    // Verify no advance option available
  });

  test('E.3.7 - API endpoint validates checklist', async () => {
    // POST /api/v1/kpi/tradesman/{id}/phase/advance
    // With incomplete checklist
    // Verify 400 error returned
  });
});
```

E.4 Bonus Calculation - Phase 2

```
describe('Phase 2 Bonus', () => {
  test('E.4.1 - $11,000 closed = $0 bonus', async () => {
    // Tradesman at Phase 2
    // Closed sales labor value: $11,000
    // Verify bonus_amount = 0
  });

  test('E.4.2 - $13,000 closed = $300 bonus', async () => {
    // Closed: $13,000
    // Verify bonus_amount = 300
  });

  test('E.4.3 - $15,000 closed = $500 bonus', async () => {
    // Closed: $15,000
    // Verify bonus_amount = 500
  });

  test('E.4.4 - $17,000 closed = $750 bonus', async () => {
    // Closed: $17,000
    // Verify bonus_amount = 750
  });

  test('E.4.5 - $20,000 closed = $1,000 bonus', async () => {
    // Closed: $20,000
    // Verify bonus_amount = 1000
  });

  test('E.4.6 - Uses labor value only', async () => {
    // Total closed: $25,000 (includes $10,000 materials)
    // Labor value: $15,000
    // Verify bonus based on $15,000
```

```
  });
});
```

E.5 Bonus Calculation - Phase 3

```
describe('Phase 3 Bonus', () => {
  test('E.5.1 - $11,000 closed = $0 bonus', async () => {
    // Tradesman at Phase 3
    // Verify bonus_amount = 0
  });

  test('E.5.2 - $14,000 closed = $400 bonus', async () => {
    // Verify bonus_amount = 400
  });

  test('E.5.3 - $17,000 closed = $700 bonus', async () => {
    // Verify bonus_amount = 700
  });

  test('E.5.4 - $20,000 closed = $1,000 bonus', async () => {
    // Verify bonus_amount = 1000
  });

  test('E.5.5 - $25,000 closed = $1,500 bonus', async () => {
    // Verify bonus_amount = 1500
  });
});
```

E.6 Bonus Period Management

```
describe('Bonus Period Management', () => {
  test('E.6.1 - Current period bonus projection', async () => {
    // GET /api/v1/kpi/bonus/current
    // Verify returns: current closed value, projected bonus
  });

  test('E.6.2 - Bonus updates in real-time', async () => {
    // Close additional sales
    // Refresh bonus projection
    // Verify updated amount
  });

  test('E.6.3 - Period closes fortnightly', async () => {
    // Trigger bonus_period_close job
    // Verify tradesman_bonus_periods record created
  });
```

```
  test('E.6.4 - Period record includes all data', async () => {
    // Verify record has: user_id, period_start, period_end,
    // closed_sales_labor_value, sales_phase, bonus_tier, bonus_amount
  });

  test('E.6.5 - Owner can approve bonus', async () => {
    // POST /api/v1/kpi/bonus/{id}/approve
    // Verify approved_by, approved_at set
  });

  test('E.6.6 - Owner can mark as paid', async () => {
    // POST /api/v1/kpi/bonus/{id}/mark-paid
    // Verify paid=true, paid_at set
  });

  test('E.6.7 - Bonus history viewable', async () => {
    // GET /api/v1/kpi/bonus/history
    // Verify returns past periods
  });

  test('E.6.8 - Non-owner cannot approve', async () => {
    // Log in as tradesman
    // Attempt to approve
    // Verify access denied
  });
});
```

E.7 Bonus Dashboard Display

```
describe('Bonus Dashboard Display', () => {
  test('E.7.1 - Tradesman sees current projection', async () => {
    // Log in as tradesman
    // Verify bonus projection displayed on dashboard
  });

  test('E.7.2 - Progress to next tier shown', async () => {
    // At $13,500 (in $300 tier)
    // Verify shows: "$500 to reach $500 bonus tier"
  });

  test('E.7.3 - Bonus history visible to tradesman', async () => {
    // Verify can see own past bonuses
  });

  test('E.7.4 - Owner sees all pending approvals', async () => {
    // Log in as owner
    // Verify list of bonuses awaiting approval
```

```
  });
});
```

Integration Tests

INT.1 KPI Module Integration

```
describe('KPI Integration with Existing Modules', () => {
  test('INT.1.1 - Job completion updates daily KPI', async () => {
    // Complete a job with logged hours
    // Verify kpi_daily_snapshot updated
  });

  test('INT.1.2 - Quote acceptance updates close rate', async () => {
    // Accept a quote
    // Verify close rate recalculated
  });

  test('INT.1.3 - Invoice payment updates cash tracking', async () => {
    // Mark invoice paid
    // Verify cash position updates
  });

  test('INT.1.4 - Schedule changes update capacity', async () => {
    // Add scheduled job
    // Verify capacity percentages update
  });

  test('INT.1.5 - Time logging updates profitability', async () => {
    // Log hours on job
    // Verify job profitability recalculates
  });

  test('INT.1.6 - User role affects dashboard access', async () => {
    // Different roles see different dashboards
    // Verify correct dashboard per role
  });
});
```

INT.2 Event Flow Tests

```
describe('Event Flows', () => {
  test('INT.2.1 - Daily revenue tracking flow', async () => {
    // Complete task with hours
    // Verify: snapshot updates → threshold check → alert if needed → dashboard reflects
```

```
  });

  test('INT.2.2 - Quote close rate flow', async () => {
    // Quote accepted
    // Verify: close rate recalculated → weekly snapshot updates → alert if low
  });

  test('INT.2.3 - Phase progression flow', async () => {
    // Complete all checklist items
    // Click advance
    // Verify: phase updates → log created → notification sent → dashboard reflects
  });

  test('INT.2.4 - Bonus calculation flow', async () => {
    // Close sales through the week
    // Period closes
    // Verify: bonus calculated → record created → owner notified → approval workflow
  });
});
```

INT.3 Full KPI Workflow

```
describe('Full KPI Workflow', () => {
  test('INT.3.1 - Complete week lifecycle', async () => {
    // Monday: Log 8 hours, $2,200 labor
    // Tuesday: Log 7 hours, $1,800 labor
    // Wednesday: Log 8 hours, $2,500 labor
    // Thursday: Log 7.5 hours, $2,000 labor
    // Friday: Log 8 hours, $2,100 labor, close 2 quotes
    //
    // Verify:
    // - Daily snapshots created each day
    // - Weekly snapshot created Friday
    // - Close rate calculated
    // - Weekly target met (> $10,000)
    // - Report generated
  });

  test('INT.3.2 - Month end rollup', async () => {
    // Create 4 weeks of data
    // Trigger monthly snapshot
    // Verify:
    // - Monthly totals correct
    // - Profit calculated
    // - Variance from target shown
    // - Monthly report generated
  });
```

```
  test('INT.3.3 - Phase progression journey', async () => {
    // New tradesman starts at Phase 1
    // 4 weeks of hitting targets
    // Complete coaching
    // Owner approves advancement
    // Verify:
    // - Checklist tracks progress
    // - Advancement works
    // - Phase 2 bonus structure now applies
  });
});
```

Performance Tests

```
describe('KPI Performance', () => {
  test('PERF.1 - Dashboard loads under 3 seconds', async () => {
    // Measure dashboard load time
    // Verify under 3000ms
  });

  test('PERF.2 - Daily snapshot job completes under 30 seconds', async () => {
    // With 10 tradesmen
    // Verify job completes quickly
  });

  test('PERF.3 - Historical data query efficient', async () => {
    // Query 12 months of monthly snapshots
    // Verify response under 2 seconds
  });

  test('PERF.4 - Calculations handle large datasets', async () => {
    // 1000+ jobs in system
    // Verify profitability calculations still fast
  });
});
```

Replit Prompt

Copy and paste this entire prompt into Replit:

I need you to run the KPI Module test suite for the Plaza Works Job Management App.

The KPI Module (Phases A-E) has been implemented. Please test all functionality.

SETUP:
1. Ensure test users exist with KPI fields populated:
   - plumber1: hourly_cost_loaded=$84.59, daily_cost_loaded=$636.44, sales_phase=1
   - plumber2: hourly_cost_loaded=$93.67, daily_cost_loaded=$702.50, sales_phase=2
   - apprentice: hourly_cost_loaded=$53.97, daily_cost_loaded=$404.80
2. Ensure kpi_targets has seed data for 1P+1A, 2P+2A, 3P+2A configurations
3. Create test jobs, quotes, and invoices for calculation testing

TEST SUITES TO RUN:

PHASE A - Database:
- Verify all 8 KPI tables exist with correct columns
- Verify user table has new KPI fields
- Verify seed data present

PHASE B - Calculations:
- Daily labor revenue calculation (per tradesman, aggregation, date filtering)
- Weekly labor revenue calculation (sum of daily, target met)
- Close rate calculation (formula, per tradesman, edge cases)
- Job profitability calculation (revenue - costs, fully loaded rates, multiple staff)
- Duration variance calculation (actual vs quoted)
- Schedule capacity calculation (per period, per tradesman, team aggregate)
- All API endpoints return correct data

PHASE C - Dashboards:
- Owner dashboard (daily/weekly/monthly views, per-tradesman cards, status indicators)
- Tradesman dashboard (own data only, progress bar, heat map, streak counter)
- Apprentice dashboard (hours tracking, simplified view)
- Marketing dashboard (lead metrics, cost thresholds)
- Mobile responsiveness

PHASE D - Alerts and Reports:
- All alert triggers fire at correct thresholds
- Alert acknowledgment works
- All scheduled jobs run correctly
- Daily, weekly, monthly reports generate
- Reports contain correct data
- Export to PDF/CSV works

PHASE E - Phase and Bonus:
- Phase progression display
- Checklist functionality
- Phase advancement workflow
- Phase 2 bonus calculations (verify all 5 tiers)
- Phase 3 bonus calculations (verify all 5 tiers)
- Bonus period management

- Approval workflow

INTEGRATION:
- KPI updates when jobs/quotes/invoices change
- Event flows work end-to-end
- Full week and month lifecycles

CRITICAL CALCULATIONS TO VERIFY:
1. $13,000 labor closed at Phase 2 = $300 bonus
2. $17,000 labor closed at Phase 3 = $700 bonus
3. Daily labor < $1,500 triggers warning alert
4. Close rate < 50% for 2 weeks triggers alert
5. Job profit = labor revenue - (hours × hourly_cost_loaded)

FOR EACH TEST:
1. Run the test
2. Record PASS or FAIL
3. If FAIL, capture the error and actual vs expected values
4. Continue to next test

AFTER ALL TESTS:
Generate a report showing:
- Total tests: [count]
- Passed: [count]
- Failed: [count]
- Pass rate: [percentage]
- List of failures with details
- Suggested fixes for failures

Please run all tests now.

Simpler Manual Testing Prompt

If the full automated suite is too complex:

Please manually test the KPI Module by verifying these critical features:

1. DAILY DASHBOARD: Log in as owner, verify you see per-tradesman daily metrics
2. CALCULATIONS: Create a job with $2,500 labor and 10 hours logged. Verify:
   - Labor cost = 10 × hourly rate
   - Profit = $2,500 - labor cost
3. ALERTS: Create a tradesman with only $1,400 labor today. Verify warning alert triggers
4. WEEKLY ROLLUP: Check that weekly totals sum daily values correctly
5. CLOSE RATE: Create 10 quotes, accept 7. Verify close rate shows 70%
6. CAPACITY: Schedule 3 days this week. Verify capacity shows 60% (3/5)

7. PHASE CHECKLIST: View a tradesman's phase progression checklist
8. BONUS CALC: Tradesman at Phase 2 closes $13,500 labor. Verify $300 bonus
9. REPORTS: Trigger daily report generation. Verify it includes all tradesmen
10. MOBILE: View tradesman dashboard on phone. Verify it's usable

For each test, report:
- What you tested
- PASS or FAIL
- Any errors or unexpected results